

From the linear model to the MLP

UE ISD-1020 · Machine Learning / Deep Learning

Stéphane Rivaud

Université Paris-Saclay · Master ISD (apprenticeship)

21 September 2026

This afternoon

- **UE ISD-1020** “Machine Learning / Deep Learning”
- Master Informatique, **ISD** track, apprenticeship
- Université Paris-Saclay · **PUIO B210**
- Teaching team: Stéphane Rivaud, Solal Nathan and Dylan Séchet
- Today: **13:30–17:00** (morning 09:00–12:30)

AM · HGB
PR-AUC 0.805



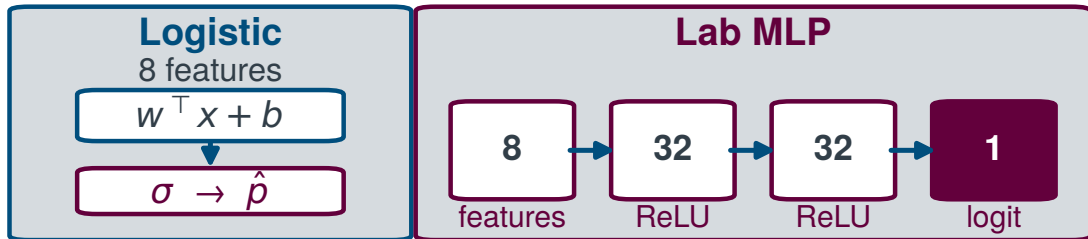
PM · MLP
first network

Boosting was the floor; now a network.

From linear to the perceptron

Definition

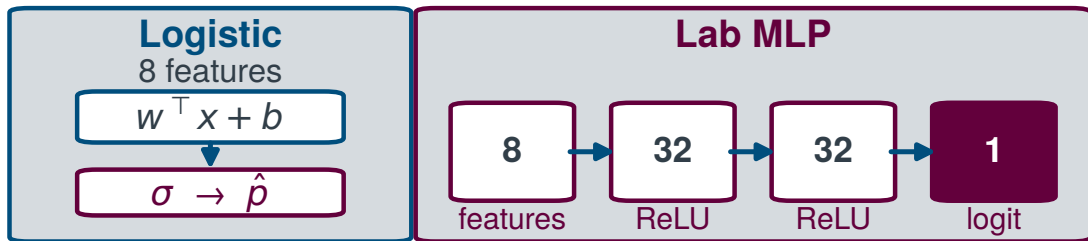
Logistic regression is $\hat{p} = \sigma(w^\top x + b)$: one hyperplane, then a sigmoid. The morning logistic **is** a network — one layer, no hidden nonlinearity. x is already chosen (8 features).



A hidden layer and the lab MLP

Definition

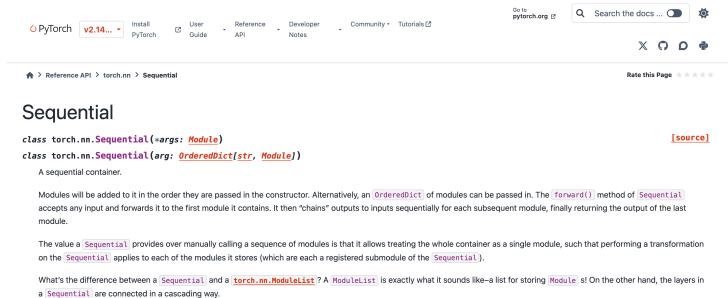
$h = \text{ReLU}(W_1x + b_1)$, then a logit $w_2^\top h + b_2$. $\text{ReLU}(z) = \max(z, 0)$ breaks linearity; stacked W s without it remain a linear model. Lab MLP: $8 \rightarrow 32 \rightarrow 32 \rightarrow 1$ logit.



The MLP of this lab

Definition

Two hidden ReLU layers, width 32, one logit — no sigmoid at the output. Dropout(p) after each ReLU ($p = 0$: off). BCEWithLogitsLoss applies σ .



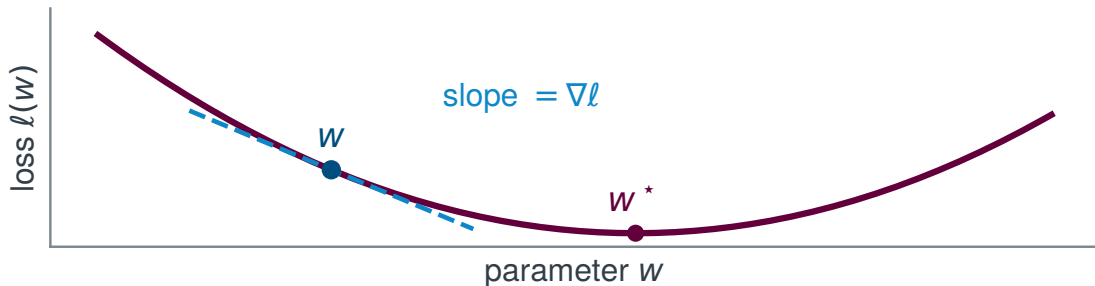
The screenshot shows the PyTorch documentation for `torch.nn.Sequential`. At the top, there's a navigation bar with links for PyTorch, Install PyTorch, User Guide, Reference API, Developer Notes, Community, and Tutorials. A search bar is also present. Below the navigation bar, the breadcrumb trail reads: Home > Reference API > torch.nn > Sequential. The main heading is "Sequential". Below it, the class definition is shown: `class torch.nn.Sequential(*args: Module)` and `class torch.nn.Sequential(arg: OrderedDict(str, Module))`. A link to the source code is provided. The description states: "A sequential container. Modules will be added to it in the order they are passed in the constructor. Alternatively, an `OrderedDict` of modules can be passed in. The `forward()` method of `Sequential` accepts any input and forwards it to the first module it contains. It then "chains" outputs to inputs sequentially for each subsequent module, finally returning the output of the last module." It then explains that the value a `Sequential` provides over manually calling a sequence of modules is that it allows treating the whole container as a single module, such that performing a transformation on the `Sequential` applies to each of the modules it stores (which are each a registered submodule of the `Sequential`). Finally, it clarifies the difference between a `Sequential` and a `torch.nn.ModuleList`: A `ModuleList` is exactly what it sounds like—a list for storing `Module`s! On the other hand, the layers in a `Sequential` are connected in a cascading way.

torch.nn.Sequential

What is a gradient

Definition

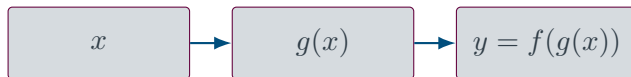
The gradient $\nabla_w \ell(w)$ is the local slope of the loss. We descend: $w \leftarrow w - \eta \nabla \ell(w)$.



The chain rule

Definition

If $y = f(g(x))$, then $\frac{dy}{dx} = f'(g(x)) g'(x)$. Each factor is evaluated **where we are**. An MLP is the same product, layer after layer.



Definition

$y = wx + b$ at $x = 2, w = 3, b = 1: y = 7. \partial y / \partial w = x = 2, \quad \partial y / \partial b = 1.$

Live demo

`requires_grad=True`, then `y.backward()`. The notebook must print `w.grad=2` and `b.grad=1`. An MLP is only this rule, longer.

Gradient descent and mini-batches

Definition

$\theta \leftarrow \theta - \eta \nabla_{\theta} \ell(\theta)$. Mini-batch SGD: a noisy gradient on a batch, not on all 6,400 rows. Here: batch 256, Adam $\eta = 10^{-2}$. One epoch = one full pass.

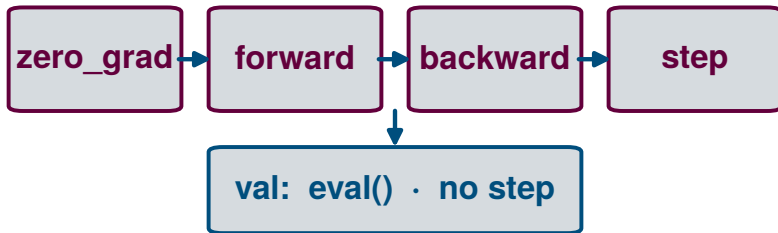
The step

Too small η : we do not move. Too large: we bounce. The lab loss is a **mean** over the examples in the batch. DataLoader shuffles (seeded) and slices.

Anatomy of the training loop

Definition

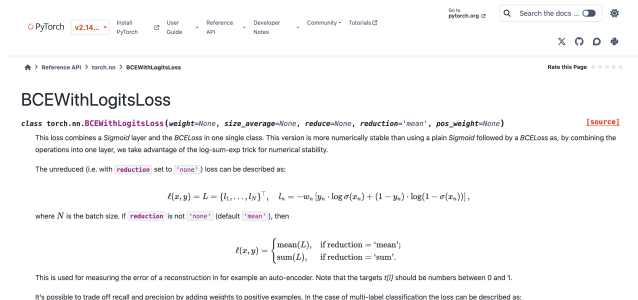
Four calls: `zero_grad` → `forward` → `backward` → `step`. Val: `model.eval()` (dropout off), no `step`.



Training loss: weighted BCE

Definition

Weighted BCE: $\text{pos_weight} = n_- / n_+ = 6183 / 217 = 28.493$ on the **train** only. Without the weight the net says no (accuracy 0.966, recall 0).



The screenshot shows the PyTorch documentation page for `BCEWithLogitsLoss`. The page header includes navigation links for PyTorch v2.14, Install PyTorch, User Guide, Reference API, Developer Notes, Community, and Tutorials. A search bar is also present. The main content area is titled "BCEWithLogitsLoss" and includes the following code snippet:

```
class torch.nn.BCEWithLogitsLoss(weight=None, size_average=None, reduce=None, reduction='mean', pos_weight=None) [source]
```

The text explains that this loss combines a Sigmoid layer and the BCE loss in one single class. It is more numerically stable than using a plain Sigmoid followed by a BCE loss, as it combines the operations into one layer, taking advantage of the log-sum-exp trick for numerical stability.

The unreduced (i.e. with `reduction` set to `"none"`) loss can be described as:

$$\ell(x, y) = L = \{l_1, \dots, l_N\}^T, \quad l_n = -w_n [y_n \cdot \log \sigma(x_n) + (1 - y_n) \cdot \log(1 - \sigma(x_n))],$$

where N is the batch size. If `reduction` is not `"none"` (default `"mean"`), then

$$\ell(x, y) = \begin{cases} \text{mean}(L), & \text{if reduction = 'mean'}; \\ \text{sum}(L), & \text{if reduction = 'sum'}. \end{cases}$$

This is used for measuring the error of a reconstruction in for example an auto-encoder. Note that the targets $t[i]$ should be numbers between 0 and 1.

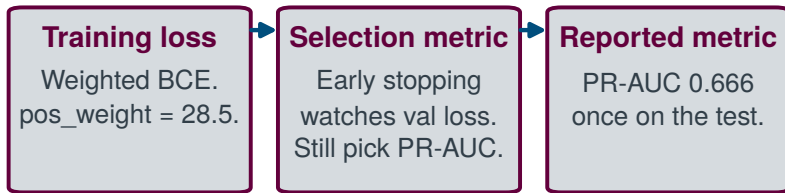
It's possible to trade off recall and precision by adding weights to positive examples. In the case of multi-label classification the loss can be described as:

`torch.nn.BCEWithLogitsLoss`

Same three lanes as this morning

Definition

`pos_weight` adjusts the **training loss**, not the selection metric. We minimize weighted BCE; we still select and report PR-AUC — the same three lanes as this morning.

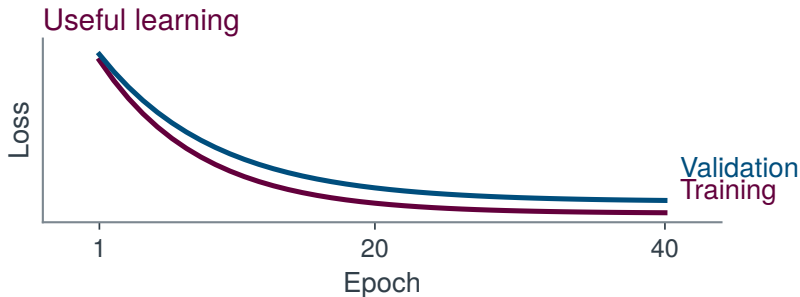


Minimize weighted BCE; still report PR-AUC — same three lanes.

Read a learning curve

Definition

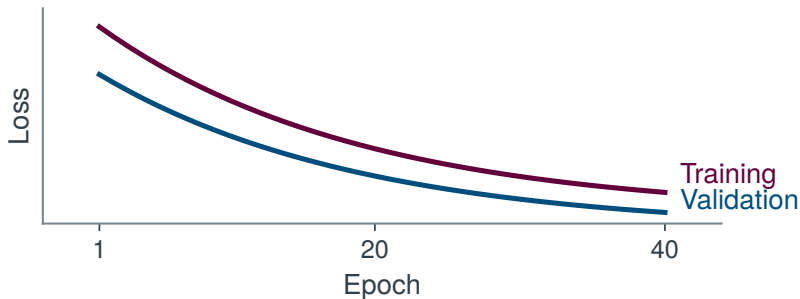
Train and val loss versus epoch. Both fall: we are learning. We choose on val; we judge the test **after** that choice.



Underfitting

Definition

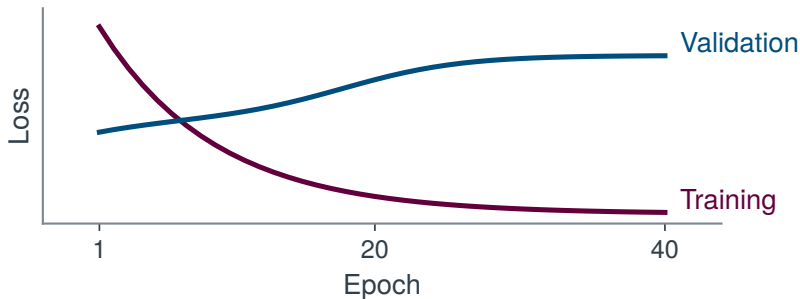
Train and val stay high: too small, too short. Width 8, 8 epochs $\rightarrow$ test PR-AUC 0.321.



Overfitting

Definition

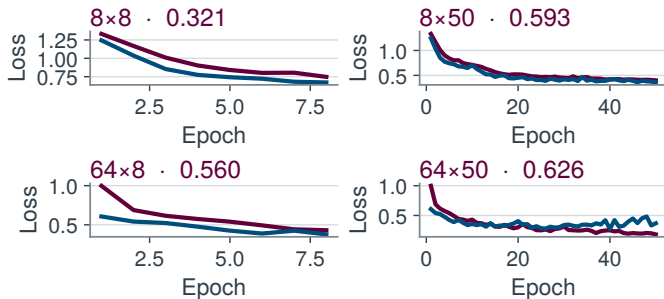
Train keeps falling; val rises or stalls. Width 64, 50 epochs: train 0.182, val 0.369.



Width $\times$ epochs (this lab)

Definition

Four CPU-friendly runs. Best train loss $\neq$ best test AP. (8, 8) underfits; (64, 50) overfits.



Three levers

Definition

Three levers, tested one by one on the MLP 32 / 40. Early stopping watches val **loss** — a selection procedure **inside** training, not a fourth metric type.

Weight decay

$\lambda = 10^{-2}$ in Adam.
Train 0.386 · val 0.315.
PR-AUC **0.697**.

Dropout

$p = 0.3$ after each ReLU.
Off in eval.
PR-AUC **0.698**.

Early stopping

Patience 8 → epoch 33.
Restore best weights.
PR-AUC 0.664 · F1 0.461.

When does a network beat boosting?

Definition

On tables of this size, trees often win without extra work. A network pays when there is volume, mixed modalities, or embeddings. Images and sequences: another regime (Monday 28 September).

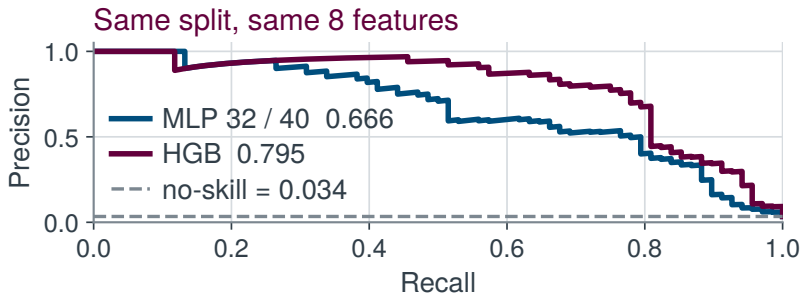
Not an API contest

Boosting is the honest floor at 10,000 rows. We are choosing a family and a cost, not a library.

MLP versus HGB, same split

Definition

Same 6,400 train / 8 features / 2,000 test. MLP 32/40: PR-AUC 0.666. HGB: 0.795.
Morning HGB 0.805 saw Type raw — Trap 2.



A Keras equivalence

This module is PyTorch

If the company uses Keras, the objects correspond. One slide, not a second library to learn.

	PyTorch	Keras
Model	<code>nn.Sequential</code>	<code>keras.Sequential</code>
Loss	<code>BCEWithLogitsLoss</code>	<code>BCE + from_logits</code>
Opt.	<code>Adam</code>	<code>keras.optimizers.Adam</code>
Fit	<code>backward; step</code>	<code>model.fit</code>

What the lab asks

1. Tensors · autograd on $y = wx + b$
2. `nn.Sequential` on standardised A14I
3. Loop + `pos_weight` + curves
4. Width $\times$ epochs grid
5. Weight decay, dropout, early stopping
6. MLP vs HGB · six protocol cells

The protocol, tonight

Definition

Step 1 keeps the **selection metric** (PR-AUC). Step 4 names the family and the **training loss** (weighted BCE).



Instantiated tonight: always-0 floor; MLP 32/40; is HGB enough?

Instructions

1. Shared course folder; **File** → **Save a copy in Drive** (new copy)
2. Default **autonomous**; a hinted variant if the room struggles
3. CPU. Allowed everywhere — you can explain every line you keep
4. “Run all” empty: **NameError**, that is intended
5. Groups of 3: you may still change until 28 September; assessed on 5 October

Attributions and licences

Module material: Stéphane Rivaud.

Tutorials

- PyTorch tutorials · BSD-3-Clause
- d2l.ai ch. 4–5 · CC BY-SA 4.0
- micrograd · MIT

Datasets

- AI4I 2020 / UCI 601 · CC BY 4.0

Documentation

- PyTorch · BSD-3-Clause

Chain rule: adapted from *Maths for DS*, lesson 6 (S. Rivaud).

PyTorch marks remain with their owners; used here only to identify the tools.